Proofs for the Alignment Kernel: Diagonal Vote and Sliding-Window Smith–Waterman

TB Hardware Accelerator project, Algorithm focus

October 2026

Abstract

This note states and proves the claims behind the proposed architecture: a diagonal vote followed by a sliding-window Smith–Waterman alignment. Each result is labelled *Proved* (a complete proof is given), *Classical* (a standard result, cited), *Computed* (an exact value obtained from the reference sequences by exhaustive calculation, with no sampling), or *Empirical* (supported only by simulated reads, and not a theorem). Section 12 lists what is not proved.

Summary of results

Result	Claim	Status
Lemma 2.1	$H(i,j)$ is the best score of a local alignment ending at (i,j)	Classical
Prop. 2.2	$0 \le H(i,j) \le M \min(i,j)$; scores fit in 9 bits	Proved
Prop. 3.1	Neighbouring cells differ by a bounded amount	Proved
Prop. 4.1	Score equals $\frac{M}{2}(m+n)$ minus a shortest-path cost	Proved
Prop. 5.1	The window computes at most mW cells	Proved
Prop. 5.3	Window score never exceeds the full score	Proved
Prop. 5.5	Window score equals the full score when an optimal path fits in the window	Proved
Theorem 6.2	The vote lands on an acceptable diagonal when substitutions are below a margin	Proved
Cor. 6.3	Then the window contains the true alignment	Proved
Tables 1, 2	Values of the margin for the three loci	Computed
Cor. 6.4	Probability bound for the vote under independent substitutions	Proved
Table 3	Storage, cell and cycle counts	Proved
Prop. 8.1	The direct vote makes exactly mn base comparisons	Proved
Prop. 8.2	The vote can be computed in $O((m+n)\log(m+n))$	Classical
Props. 9.2, 9.3	Wavefront update and comparison bounds	Proved
Prop. 10.1	Segment-vote band computes at most $mW + 2S_{\text{seg}}G$ cells	Proved
Prop. 11.1	Expected cost with a gate	Proved

1 Notation

- The read is $a = a_1 \dots a_m$ and the reference window is $b = b_1 \dots b_n$.

- Scores: match $M > 0$, mismatch $x \le M$, and $g < 0$ per gapped base. The project uses $M = 2$, $x = -1$, $g = -2$, $m = 150$ and $n \le 180$.
- $s(a_i, b_j) = M$ if $a_i = b_j$, and $s(a_i, b_j) = x$ otherwise.
- Grid *nodes* are pairs (i, j) with $0 \le i \le m$ and $0 \le j \le n$. A node with $i, j \ge 1$ is also called a *cell*.
- The *diagonal* of a node (i, j) is $j - i$.
- A *path* is a sequence of steps between nodes. A diagonal step $(i-1, j-1) \to (i, j)$ scores $s(a_i, b_j)$. A down step $(i-1, j) \to (i, j)$ and a right step $(i, j-1) \to (i, j)$ each score g. The score of a path is the sum of the scores of its steps, and the empty path scores 0. A path is a local alignment of the substrings it spans.
- $W = 2w + 1$ is the window width, w its half-width, and d_0 the diagonal chosen by the vote.

2 Smith–Waterman

The recurrence below is the local alignment algorithm of Smith and Waterman [1] with a linear gap penalty.

Definition. $H(i, 0) = H(0, j) = 0$, and for $i, j \ge 1$

$$H(i,j) = \max\big\{\, 0,\ H(i-1, j-1) + s(a_i, b_j),\ H(i-1, j) + g,\ H(i, j-1) + g \,\big\}. \tag{1}$$

The alignment score is $S = \max_{i,j} H(i, j)$.

Lemma 2.1 (Classical)**.** *$H(i, j)$ is the maximum score over all paths that end at the node (i, j), the empty path included.*

Proof sketch. Induction on $i + j$. A non-empty path ending at (i, j) ends with one of the three steps, and the rest of it is a path ending at the corresponding predecessor. Maximising over the last step and, by the induction hypothesis, over the rest gives the three non-zero terms of (1). The empty path gives the term 0. On the boundary, every non-empty path consists of gap steps only and has a negative score. □

Proposition 2.2. $0 \le H(i, j) \le M \min(i, j)$. *Consequently $0 \le S \le M \min(m, n)$, and an unsigned score needs $\lceil \log_2(M \min(m, n) + 1) \rceil$ bits.*

Proof. The lower bound is the term 0 in (1). For the upper bound use Lemma 2.1. A diagonal step increases both coordinates by one, so a path ending at (i, j) contains at most $\min(i, j)$ diagonal steps. Each scores at most M, and every other step scores $g < 0$. □

With the project values, $S \le 300$ and 9 bits suffice.

3 Differences between neighbouring cells

Proposition 3.1. *For all $1 \le i \le m$ and $1 \le j \le n$,*

$$g \le H(i,j) - H(i,j-1) \le M - g, \tag{2}$$

$$g \le H(i,j) - H(i-1,j) \le M - g, \tag{3}$$

$$x \le H(i,j) - H(i-1,j-1) \le M. \tag{4}$$

Proof. Lower bounds. The recurrence (1) gives $H(i,j) \geq H(i,j-1) + g$, $H(i,j) \geq H(i-1,j) + g$, and $H(i,j) \geq H(i-1,j-1) + s(a_i, b_j) \geq H(i-1,j-1) + x$.

Upper bounds. First extend (2) and (3) to the boundary. For $i = 0$ and $j \geq 1$ we have $H(0,j) - H(0,j-1) = 0$, and for $j = 0$ and $i \geq 1$ we have $H(i,0) - H(i-1,0) = 0$. Since $g \leq 0 \leq M - g$, both inequalities hold there as well.

Now argue by induction on $i+j$. Fix $i, j \geq 1$ and assume the three upper bounds hold at every node with a smaller coordinate sum. Consider which term attains the maximum in (1).

Case 0: $H(i,j) = 0$. All three differences are at most 0, because $H \geq 0$ everywhere, and $0 \leq M \leq M - g$.

Case 1: $H(i,j) = H(i-1,j-1) + s(a_i, b_j)$. Then $H(i,j) \leq H(i-1,j-1) + M$, which is (4). The extended lower bound (3) at $(i,j-1)$ gives $H(i-1,j-1) \leq H(i,j-1) - g$, hence $H(i,j) \leq H(i,j-1) + M - g$, which is (2). The extended lower bound (2) at $(i-1,j)$ gives $H(i-1,j-1) \leq H(i-1,j) - g$, hence $H(i,j) \leq H(i-1,j) + M - g$, which is (3).

Case 2: $H(i,j) = H(i-1,j) + g$. Then (3) holds because $g \leq M - g$. By the induction hypothesis (2) at $(i-1,j)$, or its boundary form, $H(i-1,j) \leq H(i-1,j-1) + M - g$, so $H(i,j) \leq H(i-1,j-1) + M$, which is (4). Combining this with $H(i-1,j-1) \leq H(i,j-1) - g$ gives (2).

Case 3: $H(i,j) = H(i,j-1) + g$. Symmetric to Case 2, with the roles of i and j exchanged. □

Corollary 3.2. *A side-by-side difference* (2) *or* (3) *takes one of* $M - 2g + 1$ *integer values and needs* $\lceil \log_2(M - 2g + 1) \rceil$ *bits. A same-diagonal difference* (4) *takes one of* $M - x + 1$ *values and needs* $\lceil \log_2(M - x + 1) \rceil$ *bits.*

With the project values these are the ranges $-2, \ldots, +4$ (3 bits) and $-1, \ldots, +2$ (2 bits). The simulated reads showed exactly these ranges.

Remark. Proposition 3.1 concerns the full matrix. In a window, a cell at the edge has a neighbour outside the window that is read as 0, and the bound does not apply to that pair. The simulation measured only pairs with both cells inside the window.

4 Score as a shortest-path cost

For a path P from (i_0, j_0) to (i_1, j_1) let μ, ν and γ be its numbers of matches, mismatches and gap steps, and let

$$u = i_0 + (m - i_1) + j_0 + (n - j_1)$$

be the number of bases of a and b that P leaves unaligned. Define the cost

$$c(P) = (M - x)\,\nu + \Big(\frac{M}{2} - g\Big)\gamma + \frac{M}{2}\,u. \tag{5}$$

All three coefficients are non-negative.

Proposition 4.1. *For every path* P, $\mathrm{score}(P) = \frac{M}{2}(m+n) - c(P)$. *Therefore*

$$S = \frac{M}{2}(m+n) - \min_P c(P).$$

Proof. A diagonal step advances both coordinates and a gap step advances one of them, so $(i_1 - i_0) + (j_1 - j_0) = 2(\mu + \nu) + \gamma$, and hence $m + n = 2(\mu + \nu) + \gamma + u$. Substituting into (5),

$$\begin{aligned} \frac{M}{2}(m+n) - c(P) &= M\mu + M\nu + \frac{M}{2}\gamma + \frac{M}{2}u - (M - x)\nu - \frac{M}{2}\gamma + g\gamma - \frac{M}{2}u \\ &= M\mu + x\nu + g\gamma, \end{aligned}$$

which is the score of P. The cost is therefore a decreasing function of the score over the same set of paths, the empty path included, so the maximum score corresponds to the minimum cost. □

With the project values the costs are 0 (match), 3 (mismatch), 3 (gapped base) and 1 (unaligned base), and $S = m + n - c_{\min}$. This is the quantity a time-coded ("first arrival") circuit would compute. The identity was also checked numerically on 400 random pairs.

5 The sliding window

Definition. The *window* is the set of cells

$$\mathcal{W} = \big\{(i,j) : 1 \le i \le m,\ 1 \le j \le n,\ |j - i - d_0| \le w\big\}.$$

$H_{\mathcal{W}}$ is defined by the recurrence (1) on the cells of $\mathcal{W}$ and is 0 at every other node. The window score is $S_{\mathcal{W}} = \max H_{\mathcal{W}}$.

Proposition 5.1 (Work)**.** $|\mathcal{W}| \le mW$. *The window therefore computes at most a fraction W/n of the mn cells of the full matrix.*

Proof. For each i, the condition $|j - i - d_0| \le w$ allows at most $2w + 1 = W$ values of j. □

For $m = 150$, $n = 180$ and $W = 17$ this is 2,550 against 27,000 cells, or 9.4%.

Call a path *admissible* if every node on it, except possibly its start node, is a cell of $\mathcal{W}$.

Lemma 5.2. *For a cell $(i,j) \in \mathcal{W}$, $H_{\mathcal{W}}(i,j)$ is the maximum score over admissible paths ending at (i,j), the empty path included.*

Proof. Induction on $i + j$. Let p be a predecessor of (i,j) and let σ be the score of the step from p to (i,j).

If $p \notin \mathcal{W}$, the recurrence uses $H_{\mathcal{W}}(p) = 0$ and the candidate value is σ. That is the score of the one-step path from p, which is admissible because p is its start node. No longer admissible path passes through p, because p would then be a node other than the start node that lies outside $\mathcal{W}$.

If $p \in \mathcal{W}$, the candidate value is $H_{\mathcal{W}}(p) + \sigma$. By the induction hypothesis $H_{\mathcal{W}}(p)$ is the best score of an admissible path ending at p, and appending the step keeps the path admissible. Conversely, an admissible path whose last step comes from p is an admissible path ending at p followed by that step.

Taking the maximum over the three predecessors and the empty path gives the recurrence. □

Proposition 5.3. $S_{\mathcal{W}} \le S$.

Proof. By Lemmas 2.1 and 5.2, $S_{\mathcal{W}}$ is a maximum over a subset of the paths whose maximum is S. □

Lemma 5.4. *The diagonals of the nodes of a path form a set of consecutive integers $\{k_{\text{lo}}, \dots, k_{\text{hi}}\}$, and $k_{\text{hi}} - k_{\text{lo}} \le \gamma$, the number of gap steps of the path.*

Proof. A diagonal step leaves the diagonal unchanged, a right step raises it by 1 and a down step lowers it by 1. A sequence of integers that changes by at most 1 at each step takes every value between its minimum and its maximum, and its range is at most the number of steps that change it. □

Proposition 5.5 (Exactness)**.** *Suppose $S > 0$ and some optimal path P^* has diagonal range $[k_{\mathrm{lo}}, k_{\mathrm{hi}}]$ with*

$$k_{\mathrm{hi}} - w \;\le\; d_0 \;\le\; k_{\mathrm{lo}} + w. \tag{6}$$

Then $S_{\mathcal{W}} = S$. In particular this holds if d_0 is one of the diagonals of P^* and P^* has at most w gap steps.

Proof. An optimal path with positive score does not begin with a gap step, since deleting that step would raise the score by $-g > 0$. So P^* begins with a diagonal step, and every node after its start node has $i \ge 1$ and $j \ge 1$; that is, it is a cell. By (6) every diagonal k of P^* satisfies $|k - d_0| \le w$, so those cells lie in $\mathcal{W}$ and P^* is admissible. Lemma 5.2 gives $S_{\mathcal{W}} \ge \mathrm{score}(P^*) = S$, and Proposition 5.3 gives equality.

For the last sentence, if $d_0 \in [k_{\mathrm{lo}}, k_{\mathrm{hi}}]$ and $\gamma \le w$, then Lemma 5.4 gives $k_{\mathrm{hi}} - k_{\mathrm{lo}} \le w$, and therefore $k_{\mathrm{hi}} - w \le k_{\mathrm{lo}} \le d_0 \le k_{\mathrm{hi}} \le k_{\mathrm{lo}} + w$. □

If $S = 0$ then $S_{\mathcal{W}} = 0$ as well, because $0 \le S_{\mathcal{W}} \le S$.

Remark. Proposition 5.5 is conditional. Whether the vote produces a d_0 that satisfies its hypothesis is the subject of Section 6.

6 The diagonal vote

In this section indices start at 0. The read is $r_0 \dots r_{m-1}$ and the window is $b_0 \dots b_{n-1}$. Diagonal k pairs r_i with b_{i+k}, which is the same numbering of diagonals as before.

Definition (Vote)**.** For $-(m-1) \le k \le n-1$ let

$$I(k) = \{\, i : 0 \le i < m,\ 0 \le i + k < n \,\}, \qquad N(k) = |I(k)|,$$
$$V_r(k) = \big|\{\, i \in I(k) : r_i = b_{i+k} \,\}\big|, \qquad Z_r(k) = (1+\lambda)\, V_r(k) - \lambda\, N(k),$$

for a fixed penalty $\lambda \ge 0$. $N(k)$ is the number of compared positions and depends only on k, m and n. $Z_r(k)$ equals the number of matches minus λ times the number of mismatches. The vote returns a diagonal d_0 that maximises Z_r. With $\lambda = 0$ it counts matches; with $\lambda = 1$ it is matches minus mismatches.

Definition (Read model)**.** Let E be the reference genome around the window, with $b_j = E[\omega + j]$. A *template* $t_0 \dots t_{m-1}$ is given by a start p and at most one indel of length g at read position q:

- no indel: $t_i = E[p + i]$;
- deletion: $t_i = E[p + i]$ for $i < q$, and $t_i = E[p + g + i]$ for $i \ge q$;
- insertion: $t_i = E[p + i]$ for $i < q$, t_i is unconstrained for $q \le i < q + g$, and $t_i = E[p - g + i]$ for $i \ge q + g$.

Let F be the set of unconstrained positions, which is empty unless there is an insertion. A read r *follows the template with at most e substitutions* if $r_i = t_i$ for all $i \notin F$ except at most e positions.

The *true diagonals* are $k_1 = p - \omega$ for the part before the indel, and $k_2 = k_1$ (no indel), $k_2 = k_1 + g$ (deletion) or $k_2 = k_1 - g$ (insertion) for the part after it. Write $\kappa_{\mathrm{lo}} = \min(k_1, k_2)$ and $\kappa_{\mathrm{hi}} = \max(k_1, k_2)$. The *true alignment* T pairs each read base with the reference base it was copied from, wherever that base lies in the window. All its diagonals lie in $[\kappa_{\mathrm{lo}}, \kappa_{\mathrm{hi}}]$.

A diagonal d is *acceptable* if the window centred on it contains every true diagonal:

$$\mathcal{A} = \{\, d : \kappa_{\mathrm{hi}} - w \le d \le \kappa_{\mathrm{lo}} + w \,\}.$$

For the template alone define the pessimistic and optimistic match counts

$$V^-(k) = \big|\{\, i \in I(k) \setminus F : t_i = b_{i+k} \,\}\big|, \qquad V^+(k) = V^-(k) + |I(k) \cap F|,$$

and $Z^{\pm}(k) = (1+\lambda)V^{\pm}(k) - \lambda N(k)$. The *margin* of the template is

$$\Delta_\lambda = \max_{k \in \mathcal{A}} Z^-(k) \;-\; \max_{k \notin \mathcal{A}} Z^+(k). \tag{7}$$

Here k ranges over $-(m-1), \dots, n-1$. We assume $g \le 2w$, so that $\mathcal{A}$ is not empty. If every diagonal is acceptable, the second maximum is over an empty set and is taken to be $-\infty$. The margin depends only on E, the window, m, w, λ and the template parameters (p, q, g, type). It can be computed before any read is seen.

Lemma 6.1. *If r follows the template with at most e substitutions, then for every k*

$$V^-(k) - e \;\le\; V_r(k) \;\le\; V^+(k) + e.$$

Proof. Split $I(k)$ into three sets. On positions outside F where $r_i = t_i$, the read and the template give the same indicator. On the at most e substituted positions, each indicator changes by at most 1. On positions in F the read contributes between 0 and 1 each, which lies between the contribution counted in V^- (none) and in V^+ (all). □

Theorem 6.2 (Vote)**.** *If r follows the template with at most e substitutions and*

$$2\,(1+\lambda)\,e \;<\; \Delta_\lambda, \tag{8}$$

then every maximiser of Z_r is acceptable.

Proof. Let $k_A \in \mathcal{A}$ attain $\max_{k \in \mathcal{A}} Z^-(k)$. By Lemma 6.1, $Z_r(k_A) \ge Z^-(k_A) - (1+\lambda)e$. For any $k \notin \mathcal{A}$, Lemma 6.1, the definition (7) and the hypothesis (8) give

$$Z_r(k) \le Z^+(k) + (1+\lambda)e \le Z^-(k_A) - \Delta_\lambda + (1+\lambda)e < Z^-(k_A) - (1+\lambda)e \le Z_r(k_A).$$

So no diagonal outside $\mathcal{A}$ maximises Z_r. □

Corollary 6.3 (The window contains the true alignment)**.** *Under the hypothesis of Theorem 6.2, the window centred on d_0 contains every diagonal of the true alignment T, and*

$$\text{score}(T) \;\le\; S_{\mathcal{W}} \;\le\; S.$$

Proof. $d_0 \in \mathcal{A}$ means $d_0 - w \le \kappa_{\text{lo}}$ and $\kappa_{\text{hi}} \le d_0 + w$, so every diagonal of T is within w of d_0. Take T restricted to the grid, starting at its first paired base. Its nodes after the start node are cells on those diagonals, so the path is admissible, and Lemma 5.2 gives $\text{score}(T) \le S_{\mathcal{W}}$. The second inequality is Proposition 5.3. □

Remark. This is the guarantee that matters for calling mutations: the window cannot lose the alignment the read actually came from. It does not assert $S_{\mathcal{W}} = S$; see Section 12.

Table 1: Minimum margin Δ_λ over all templates, and the largest tolerated number of substitutions e.

		$\lambda = 0$		$\lambda = 1$	
Locus	Case	$\min \Delta$	e	$\min \Delta$	e
rpoB	no indel	56	27	106	26
rpoB	deletion ≤ 8	5	2	12	2
rpoB	insertion ≤ 8	-4	none	10	2
katG	no indel	32	15	78	19
katG	deletion ≤ 8	1	0	9	2
katG	insertion ≤ 8	-9	none	4	0
inhA	no indel	36	17	78	19
inhA	deletion ≤ 8	5	2	14	3
inhA	insertion ≤ 8	-4	none	9	2

Table 2: Share of all 782,217 templates for which the vote is guaranteed with at least e substitutions.

e	0	1	2	3	5	8
$\lambda = 0$	99.91%	99.83%	99.70%	99.50%	98.79%	96.55%
$\lambda = 1$	100%	$> 99.99\%$	$> 99.99\%$	99.95%	98.99%	92.61%

6.1 Margins for the three loci (Computed)

The script `reference/vote_margin.py` evaluates Δ_λ exactly for $m = 150$, $w = 8$, every read start for which the read covers the hotspot centre with at least 20 bp on each side (111 starts per locus), every indel position, every length $g = 1, \ldots, 8$ and both indel types. That is 260,739 templates per locus and 782,217 in total. The windows are 180 bp for rpoB and 120 bp for katG and inhA, taken from NC_000962.3. The window coordinates are provisional.

Table 1 gives the minimum margin and the largest e that satisfies (8). Table 2 gives the share of templates for which the vote is guaranteed with at least e substitutions. These shares weight every template equally; they are not probabilities over real reads.

The tables support four statements.

1. *With the plain match count ($\lambda = 0$) the vote is not guaranteed.* For some insertion templates the margin is negative, so a wrong diagonal can win even with no substitution at all. The cause is that a diagonal on which more of the read overlaps the window accumulates chance matches over more positions. The sequence is GC-rich, and about 40% of unrelated positions match.

2. *With $\lambda = 1$ every template has a positive margin.* A read with no substitutions always votes for an acceptable diagonal, for every start and every single indel of up to 8 bp at all three loci.

3. *Reads without an indel are safe by a wide margin* under either statistic: at least 15 substitutions are tolerated.

4. *Indel reads have thin worst-case margins.* A few templates tolerate no substitution, or only two. The bound is adversarial: it assumes that every substitution removes a match from the true diagonal and adds one to the same wrong diagonal. It is therefore conservative.

The hardware cost of $\lambda = 1$ is small. $Z(k) = 2V(k) - N(k)$, and $N(k)$ is a function of the shift only, so it needs one counter and one subtraction.

Corollary 6.4 (Independent substitutions)**.** *Suppose each of the m read bases is substituted independently with probability ε, and the template tolerates e^* substitutions. Then*

$$\Pr[\textit{the vote is not acceptable}] \;\le\; \Pr\big[\mathrm{Bin}(m,\varepsilon) > e^*\big] = \sum_{i>e^*} \binom{m}{i} \varepsilon^i (1-\varepsilon)^{m-i}.$$

Proof. By Theorem 6.2, the vote can fail only if more than e^* substitutions occur. □

For $m = 150$ the bound evaluates as follows.

e^*	Applies to	$\varepsilon = 0.005$	$\varepsilon = 0.01$	$\varepsilon = 0.02$
19	every read without an indel, $\lambda = 1$	1.9×10^{-22}	1.0×10^{-16}	3.1×10^{-11}
3	99.95% of templates, $\lambda = 1$	7.1×10^{-3}	6.5×10^{-2}	3.5×10^{-1}

The second row is weak because the bound is adversarial, not because the vote fails that often. Here ε must include true SNPs as well as sequencing errors.

Empirical check. On 16,500 simulated reads (SNPs, 0.5% errors, indels of up to 8 bp), the $W = 17$ window disagreed with full Smith–Waterman on 1 read with either statistic, although the two statistics chose different diagonals on 143 reads. The change to $\lambda = 1$ therefore buys a guarantee; it showed no measurable difference in simulation.

7 Storage, cells and cycles

Table 3 follows by counting.

Table 3: Hardware quantities for $m = 150$, $n = 180$, $W = 17$.

Quantity	Full SW, systolic	Window SW	Diagonal vote, streaming
DP cells in hardware	$\min(m,n) = 150$	$W = 17$	none (m two-bit comparators)
Cycles per read	$m + n - 1 = 329$	$m = 150$	$m + n - 1 = 329$
Traceback bits	$2mn = 54{,}000$	$2mW = 5{,}100$	none

- Without pipelining, one read takes $(m + n - 1) + m = 479$ cycles through the vote and the window. That is more than the 329 cycles of a full systolic array.
- With the two stages pipelined, throughput is set by the vote: one read per $m + n - 1$ cycles, the same as the full array.
- For the dynamic-programming part alone, the product of cells and cycles falls from about $\min(m,n)\,(m+n)$ to mW.

The design therefore reduces area and storage. It does not reduce the number of clock cycles per read.

8 Cost of the vote

Proposition 8.1. *Computing $V(k)$ for every k by direct comparison uses exactly mn base comparisons.*

Proof. $\sum_k N(k)$ counts the pairs (i, j) with $0 \le i < m$ and $0 \le j < n$, each exactly once, on its diagonal $k = j - i$. □

In a sequential model the direct vote is therefore $\Theta(mn)$, the same order as full Smith–Waterman. Its advantage there is the constant: a two-bit comparison against a cell update. With machine words of ω bits the comparisons pack into $O(mn/\omega)$ word operations.

Proposition 8.2 (Classical)**.** *All values $V(k)$ can be computed in $O\big((m+n)\log(m+n)\big)$ time.*

Proof sketch.

$$V(k) = \sum_{c \in \{\mathrm{A,C,G,T}\}} \sum_{i} [r_i = c]\,[b_{i+k} = c].$$

For each letter c the inner sum is a cross-correlation of two 0/1 vectors, which a fast Fourier transform computes for all k at once. Four transforms suffice. This is the match-counting technique of Fischer and Paterson [3]. □

Hence the vote followed by the window costs $O\big((m+n)\log(m+n) + mW\big)$ in a sequential model, which is below $\Theta(mn)$ when $W = o(n)$. The hardware does not use this method, so this bound is not a claim about the hardware.

9 The wavefront alternative

This section concerns Myers's algorithm [2] for two strings of lengths N_a and N_b. Let D be the minimum number of single-base insertions and deletions that turn one string into the other; a substitution counts as 2. Place the strings on a grid with coordinates (x, y) and number the diagonals by $k = x - y$. A horizontal step raises x and k, a vertical step raises y and lowers k, and a diagonal step along a match is free.

The algorithm works in rounds $d = 0, 1, 2, \ldots$. In round d it visits the diagonals $k = -d, -d+2, \ldots, d$. For each it takes the starting point

$$x_{\mathrm{start}}(d, k) = \max\big\{\, x(d-1, k-1) + 1,\ x(d-1, k+1) \,\big\} \tag{9}$$

and then slides along matches as far as possible, which gives $x(d, k)$. It stops when a slide reaches (N_a, N_b).

Lemma 9.1 (Classical [2])**.** *$x(d, k)$ is the largest x on diagonal k that is reachable from the origin with exactly d non-diagonal steps.*

Proof sketch. Any path to diagonal k with d non-diagonal steps makes its last such step from diagonal $k-1$ or $k+1$ after $d-1$ of them. Replacing its prefix by the furthest-reaching path on that diagonal does not move the landing point of the step backwards, and sliding along matches afterwards only moves it forwards. Induct on d. □

It follows that the algorithm stops in round D.

Proposition 9.2 (Updates)**.** *The number U of (round, diagonal) updates satisfies*

$$\frac{D(D+1)}{2} + 1 \;\le\; U \;\le\; \frac{(D+1)(D+2)}{2}.$$

Proof. Round d has $d+1$ diagonals. Rounds $0, \dots, D-1$ run in full, which is $\sum_{d=0}^{D-1}(d+1) = D(D+1)/2$ updates. Round D makes at least 1 and at most $D+1$. □

Proposition 9.3 (Comparisons)**.** *The number C of base comparisons satisfies*

$$C \;\le\; U + (2D+1)\min(N_a, N_b).$$

Proof. Each slide ends at its first mismatch or at the edge of the grid, so there is at most one failed comparison per update, and at most U in total.

A successful comparison advances the slide by one point on its diagonal. We show that no point is passed twice. Let (x, y) be the point where the slide on diagonal k stopped in round $d-2$. A vertical step from it lands on diagonal $k-1$ with the same x, so $x(d-1, k-1) \ge x$. A horizontal step lands on diagonal $k+1$ at $x+1$, so $x(d-1, k+1) \ge x+1$. If one of these steps would leave the grid, the other still gives the bound, or the starting point falls outside the grid and the slide makes no comparison. By (9), $x_{\text{start}}(d, k) \ge x+1$: each slide begins beyond the point where the previous slide on the same diagonal stopped.

At most $2D+1$ diagonals are ever visited, and a diagonal has at most $\min(N_a, N_b)$ points. □

Corollary 9.4. *The worst-case work of the wavefront algorithm is $O\big(D\min(N_a, N_b) + D^2\big)$.*

Both bounds were checked on 3,000 random pairs with no violation. For highly similar sequences the observed work is close to $N + D^2$, because almost all matches lie on one diagonal. That is an empirical observation, with an expected-case analysis in the literature. It is not a worst-case bound.

10 The segment-vote band

Split the read into consecutive segments of length at most S_{seg}. Segment t votes for a diagonal d_t. A row in segment t computes the diagonals from $\min(d_{t-1}, d_t, d_{t+1}) - w$ to $\max(d_{t-1}, d_t, d_{t+1}) + w$, where a missing neighbour is omitted. Let $G = \sum_t |d_{t+1} - d_t|$.

Proposition 10.1. *The number of cells computed is at most $mW + 2S_{\text{seg}}G$.*

Proof. A row in segment t computes at most $R_t + W$ cells, where R_t is the range of $\{d_{t-1}, d_t, d_{t+1}\}$. The range of three numbers is at most the sum of the two consecutive absolute differences, so $R_t \le |d_t - d_{t-1}| + |d_{t+1} - d_t|$. Summing over the at most S_{seg} rows of each segment and over all segments,

$$\text{cells} \;\le\; mW + S_{\text{seg}} \sum_t R_t \;\le\; mW + 2S_{\text{seg}}G,$$

since each difference $|d_{t+1} - d_t|$ appears in at most two of the R_t. □

With no indel, $G = 0$ and the count is at most mW. With one indel of length g, $G = g$. In the worst case the band covers the whole matrix.

Remark. This bounds the work only. The segment-vote band missed 7–13% of 5–12 bp indels in simulation, all within one segment length of the end of the overlap, and there is no correctness theorem for it.

11 The gate

Proposition 11.1. *If a read passes the gate with probability ρ, the vote costs C_{vote} and the window costs C_{SW}, then the expected cost per read is*

$$\mathbb{E}[T] = C_{\text{vote}} + (1 - \rho)\, C_{\text{SW}}, \qquad C_{\text{vote}} \le \mathbb{E}[T] \le C_{\text{vote}} + C_{\text{SW}}.$$

Proof. Every read pays for the vote. The window runs with probability $1 - \rho$. The claim follows by linearity of expectation. □

The value of ρ is empirical. It depends on the threshold, which has not been chosen, and on the reads.

12 What is not proved

1. $S_{\mathcal{W}} = S$ **in general.** Corollary 6.3 gives $\text{score}(T) \le S_{\mathcal{W}} \le S$. Full Smith–Waterman may find a higher-scoring alignment that leaves the window, and nothing here excludes that. In simulation it happened in about 1 read in 16,500.

2. **The vote for reads outside the model.** Theorem 6.2 covers one indel of length at most w together with substitutions. Reads with two or more indels, chimeric reads, and reads from elsewhere in the genome are not covered.

3. **The vote with $\lambda = 0$.** The margin is negative for some templates, so there is no guarantee.

4. **The margins on the final windows.** Section 6.1 used provisional window coordinates and forward-strand reads. The numbers must be recomputed once the coordinates and the strand handling are fixed.

5. **The gate.** There is no theorem about which reads may skip Smith–Waterman. In simulation, 3.6% of indel reads had an ungapped score equal to the full score because the indel was within 14 bp of the end of the overlap.

6. **The segment-vote band and the adaptive band.** Only the work bound of Proposition 10.1 is proved. The adaptive band was worse than a fixed band in simulation.

7. **Typical-case cost of the wavefront method.** Propositions 9.2 and 9.3 are worst-case bounds only.

8. **Speed and power of real hardware.** Table 3 counts cells and cycles. Clock frequency, routing and power have not been measured.

Scripts

Result	Script
Margins (Tables 1, 2)	`reference/vote_margin.py`
Difference ranges (Corollary 3.2)	`reference/ideas_study.py`
	`reference/window_study.py`
Window and vote on simulated reads	`reference/band_study.py`
Wavefront counts (Propositions 9.2, 9.3)	`reference/ideas_study.py`

References

[1] T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.

[2] E. W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1:251–266, 1986.

[3] M. J. Fischer and M. S. Paterson. String-matching and other products. In *Complexity of Computation*, SIAM–AMS Proceedings, volume 7, pages 113–125, 1974.